

“Self-balanced robot with gesture controlled with ESP8266”
A Project report submitted in the partial fulfillment the award of degree of
BACHELOR OF TECHNOLOGY
IN
Electronics and Communication Engineering
BY

P. Jagadeesh Kumar	REGNO: 231801130017
N. POORNESH	REGNO: 231801130018
R. NIKHILESH	REGNO: 231801410012
T. ROHITH	REGNO:231801410005
V. DINESH KUMAR	REGNO: 231801130012

Under the esteemed Guidance of

Dr. K. JOGI NAIDU
Assistant Professor



Centurion
UNIVERSITY

Shaping Lives...
Empowering Communities...

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT
ANDHRA PRADESH
TEKKALI VILLAGE, TEKKALI MANDAL 535003
(2026-2027)

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ANDHRA PRADESH

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**



**Centurion
UNIVERSITY**

*Shaping Lives...
Empowering Communities...*

BONAFIDE CERTIFICATE

This is to certify that the project work entitled “Self-balanced robot with gesture controlled” of project work done by P. Jagadeesh Kumar (231801130017), N. POORNESH (231801130018), R. NIKHLESH (231801410012), T. ROHITH (231801410005), V. DINESH KUMAR (231801130012). Under the esteemed Guidance of for the award the Degree of **BACHELOR OF TECHNOLOGY in ELECTRONICS AND COMMUNICATION ENGINEERING, CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT**, during the academic year **2024-2025**.

INTERNAL GUIDE

Dr. K. JOGI NAIDU
Assistant Professor
Dept. of ECE

HEAD OF THE DEPARTMENT

Dr. K. JOGI NAIDU
Assistant Professor
Dept. of ECE

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is with at most pleasure and excitement we submit our project partial fulfillment of the requirement for the award of Bachelor of Technology.

The project is a result to the cumulate efforts, support, guidance, encouragement and inspiration from many of those for whom we have to give our truthful honor and express gratitude through bringing out this project at the outset as per our knowledge.

We convey my special thanks to our project Guide **Dr. K. Jogi Naidu, Assistant Professor**, who has guided, encouraged and tremendously supported me to enhance my knowledge with present working of this project to bring out enriching the quality of project.

We express my appreciativeness to **Dr. K. Jogi Naidu**, Head of the Department of ECE, who facilitated us to providing the friendly environment which helped to enhance my skills in present project.

We thank **Dr. D. Srinivasa Rao** Ph.D, Dean of SoET, Vizianagaram Campus for their invaluable guidance, insightful feedback, and continuous support throughout the course of this project. Your expertise and mentorship have been invaluable.

We convey my sincere thanks to **Dr. P. Pallavi, Registrar of Centurion University of Technology and Management**, who provided us with an opportunity to take on project work in well-equipped laboratories of Computer Science Department in our college.

At the outset, we thank to **Prof. Prashanth Kumar Mohanty** beloved **Vice Chancellor of Centurion University of Technology and Management** who is the back bone by providing for completion of this project, Thank you sir.

DECLARATION

We hereby declare that the project entitled " Self-balanced robot with gesture controlled" submitted to the fulfilment of award the Degree of **B. TECH (ECE)** at **CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT (A.P)**. This project work in original has not been submitted so far in any part or full for any other university or institute for the award of any Degree or Diploma.

P. JAGADEESH KUMAR	REGNO: 231801130017
N. POORNESH	REGNO: 231801130018
R. NIKHLESH	REGNO: 231801410012
T. ROHITH	REGNO:231801410005
V. DINESH KUMAR	REGNO: 231801130012

ABSTRACT

The rapid advancement of robotics and embedded systems has led to the development of intelligent and interactive machines. This project focuses on the design and implementation of a self-balancing robot with gesture control, which combines stability control and wireless human interaction. The system aims to mimic the behavior of an inverted pendulum by maintaining balance on two wheels using real-time feedback.

The core functionality of the robot is achieved using MPU6050 sensors, which provide accelerometer and gyroscope data to determine the tilt angle and orientation. One sensor is mounted on the robot to maintain balance, while another is used in the gesture control unit to capture hand movements. These sensor readings are processed by microcontrollers to ensure accurate and continuous operation.

The system employs ESP32 and ESP8266 microcontrollers for processing and wireless communication. The gesture data detected by the controller unit is transmitted wirelessly to the robot using Wi-Fi. Based on the received signals, the robot interprets the direction of motion such as forward, backward, left, or right, while simultaneously maintaining its balance using control algorithms.

A control mechanism, typically based on a PID (Proportional-Integral-Derivative) algorithm, is implemented to stabilize the robot. The controller continuously adjusts motor speed and direction based on sensor feedback to correct any deviation from the vertical position. This ensures smooth and stable balancing even when external disturbances are present.

The developed system demonstrates an effective integration of sensor technology, wireless communication, and control systems. It provides a low-cost and efficient solution for real-time robotic control and can be extended for advanced applications such as autonomous robots, smart mobility systems, and human-machine interaction.

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. BACKGROUND.....	1
1.2. PROBLEM STATEMENT	2
1.3. OBJECTIVES	2
1.4. SCOPE OF THE PROJECT	3
1.5. ORGANIZATION OF THE REPORT	4
2. LITERATURE SURVEY.....	4
2.1. REVIEW OF EXISTING WORK.....	5
2.2. COMPARISON/ANALYSIS	5
2.3. RESEARCH GAP	5
3. METHODOLOGY	6
3.1. TOOLS & TECHNOLOGIES USED	7
3.2. SYSTEM DESIGN.....	7
3.3. ARCHITECTURE	8
3.4. SOFTWARE AND TOOLS:	8
3.5. WORKFLOW	9
4. IMPLEMENTATION	10
4.1. MODULES DESCRIPTION	11
4.2. ALGORITHMS/PROCESSES	11
4.3. SOURCE CODE:.....	ERROR! BOOKMARK NOT DEFINED.
5. RESULTS AND DISCUSSION.....	12
5.1. ADMIN PANEL	ERROR! BOOKMARK NOT DEFINED.
6. CONCLUSION & FUTURE SCOPE.....	13
6.1. CONCLUSION.....	15
6.2. FUTURE SCOPE.....	15
7. REFERENCE	ERROR! BOOKMARK NOT DEFINED.

INTRODUCTION

The field of robotics has experienced rapid growth due to advancements in embedded systems, control theory, and wireless communication technologies. One of the most interesting applications of these technologies is the development of self-balancing robots, which demonstrate real-time dynamic stability similar to an inverted pendulum. These robots continuously adjust their position to maintain balance on two wheels, making them an important study model in control systems and robotics.

This project focuses on the design and implementation of a self-balancing robot with gesture control, combining stability control with intuitive human interaction. The system uses MPU6050 sensors, which consist of an accelerometer and gyroscope, to measure tilt angle and orientation. Based on this data, the robot continuously calculates the required corrections to maintain an upright position.

To enable user interaction, a gesture control system is implemented using another MPU6050 sensor. The hand movements of the user are detected and converted into directional commands such as forward, backward, left, and right. These commands are transmitted wirelessly using ESP32 and ESP8266 microcontrollers, ensuring real-time communication between the controller unit and the robot.

The motion of the robot is controlled using a TB6612FNG motor driver, which efficiently drives the DC motors with low power loss and high performance. The motor driver receives control signals from the microcontroller and adjusts the speed and direction of the motors accordingly. Compared to traditional motor drivers, the TB6612FNG provides better efficiency, faster switching, and compact design, making it suitable for battery-operated robotic systems.

To maintain stability, a feedback control system based on a PID (Proportional-Integral-Derivative) algorithm is used. The controller continuously processes sensor data and adjusts motor speed to correct any deviation from the vertical position. This integration of gesture control, efficient motor driving, and balancing mechanism results in a responsive and stable robotic system.

The objective of this project is to develop a cost-effective and efficient self-balancing robot that demonstrates the integration of sensors, wireless communication, motor control, and embedded systems. This project has potential applications in robotics research, automation, and smart mobility systems.

1.1. Background

Self-balancing robots are an important application of control system engineering and are widely used to demonstrate the concept of an inverted pendulum. In such systems, maintaining stability is a challenging task because the robot must continuously adjust its position to prevent falling. This requires real-time sensing, fast processing, and precise motor control. The development of microcontrollers and MEMS-based sensors has made it possible to design compact and efficient self-balancing systems. The introduction of sensors like the MPU6050, which combines an accelerometer and gyroscope, has significantly improved motion sensing and orientation detection. These sensors provide accurate data about tilt angle and angular velocity, which are essential for balancing applications. Along with this, modern microcontrollers such as ESP32 and ESP8266 have enabled wireless communication and real-time data processing, making robotic systems more intelligent and interactive. In addition, efficient motor driving plays a crucial role in the performance of balancing robots. The TB6612FNG motor driver is widely used due to its low power consumption, high efficiency, and compact size compared to traditional drivers. By integrating these components with control algorithms like PID, it becomes possible to develop a stable and responsive self-balancing robot. This project builds upon these technologies to create a system that not only balances itself but also responds to user gestures in real time.

1.2. Problem Statement

Maintaining the upright position of a two-wheeled robot is a challenging task due to its inherently unstable nature, similar to an inverted pendulum system. The robot tends to fall if continuous and precise corrective actions are not applied in real time. Therefore, the problem is to design and implement a self-balancing robot using an ESP8266 microcontroller and MPU6050 sensor that can accurately detect tilt, process the data, and control motor movement using a suitable control algorithm to achieve stable and continuous balancing under varying conditions.

1.3. Objectives

1. Hardware Integration

- To interface the ESP8266 microcontroller with the MPU6050 sensor for measuring tilt angle and motion.
- To connect the motor driver module (TB6612FNG) with ESP8266 for controlling DC motor speed and direction.
- To integrate a rechargeable battery system to provide stable power supply to all components.

2. Firmware Configuration and Setup

- To configure ESP8266 in Arduino IDE and install required libraries.
- To initialize and calibrate MPU6050 sensor for accurate data reading.
- To implement PID control and generate PWM signals for motor control.

1.4.Scope of the Project

Inclusions:

This project includes the complete design, integration, and testing of a self-balancing robot with gesture controlled using the ESP8266 microcontroller and MPU6050 sensor and TB6612FNG. The scope covers interfacing the MPU6050 sensor for real-time measurement of tilt angle and motion, along with the integration of a motor driver module to control DC motors for balancing action. It also involves implementing a PID control algorithm to maintain stability by continuously adjusting motor speed and direction based on sensor feedback. The project includes firmware development, sensor calibration, and tuning of PID parameters to achieve smooth and stable balancing. Add The scope of this project includes the design, development, and implementation of a self-balancing robot integrated with gesture-based control using embedded systems and wireless communication. It involves the use of MPU6050 sensors for real-time orientation detection, ESP32 and ESP8266 microcontrollers for processing and communication, and the TB6612FNG motor driver for efficient motor control. The project covers the application of control algorithms such as PID for maintaining stability, along with gesture recognition for intuitive user interaction. Furthermore, it provides a foundation for future enhancements such as integration with IoT, artificial intelligence, obstacle detection, and autonomous navigation, making it applicable in areas like robotics research, smart systems, and

Exclusions:

This project does not include advanced features such as autonomous navigation, obstacle detection, or machine learning-based decision making, as the primary focus is on achieving stable balancing and basic gesture-based control. It excludes the use of high-end sensors like LiDAR or vision systems, and does not involve complex mapping or path planning algorithms. Additionally, the system is limited to short-range wireless communication using ESP modules and does not cover long-distance communication or cloud-based control. Mechanical design optimization and industrial-grade robustness are also beyond the scope of this project, as it is intended for educational and experimental purpose

Limitations:

The self-balancing robot system has certain limitations due to hardware and design constraints. The accuracy of balancing depends heavily on sensor calibration, and any noise or drift in the MPU6050 readings can affect stability. The system relies on a PID control algorithm, which requires proper tuning; incorrect tuning may lead to oscillations or instability. Wireless communication using ESP32 and ESP8266 is limited in range and may experience delays or interference, affecting real-time response. Additionally, the robot operates effectively only on flat and smooth surfaces, as uneven terrain can disturb balance. Battery limitations also restrict continuous operation time, and the system does not include advanced features such as obstacle detection or autonomous navigation.

Organization of the Report

This report is organized to provide a comprehensive understanding of the design, implementation, and evaluation of the self-balancing robot using ESP8266 and MPU6050. Chapter 1 introduces the project, including background, problem statement, objectives, scope, and limitations. Chapter 2 reviews related concepts such as self-balancing systems, control theory, and sensor technologies. Chapter 3 describes the hardware and software components, system architecture, and design methodology including sensor interfacing, motor control, and power management. Chapter 4 explains firmware development, sensor calibration, and implementation of the PID control algorithm along with parameter tuning. Chapter 5 presents testing and validation procedures to evaluate stability and performance under different conditions. Chapter 6 discusses the results, challenges faced, and overall system performance. Finally, Chapter 7 concludes the report with key findings and suggests future improvements, ensuring a clear and logical flow from concept to implementation and evaluation.

LITERATURE SURVEY

2.1 Review of Existing Work

Self-balancing robots have been widely studied as practical implementations of control system theory, particularly the inverted pendulum model. Early designs primarily relied on basic microcontrollers and analog sensors, which limited accuracy and response time. With the advancement of MEMS technology, sensors like the MPU6050 have significantly improved the measurement of tilt angle and angular velocity, enabling more precise balancing. Many existing systems implement PID control algorithms to maintain stability, while some advanced models use Kalman filters or complementary filters for better sensor fusion and noise reduction. In terms of control methods, earlier robots were operated using wired connections or simple remote controls, which restricted user interaction and flexibility. Recent developments have introduced wireless communication using Bluetooth or Wi-Fi modules, allowing remote and real-time control. However, most of these systems focus either on balancing or basic remote operation, lacking intuitive control mechanisms such as gesture-based interaction. Additionally, traditional motor drivers like L298N have been widely used but suffer from higher power losses and lower efficiency compared to modern drivers like TB6612FNG. Despite these advancements, many existing solutions are either costly, complex, or not user-friendly. Therefore, there is a need for a system that integrates efficient motor control, accurate sensing, wireless communication, and intuitive gesture-based control in a cost-effective and simplified manner, which is addressed in this project.

2.2 Comparison/Analysis

Existing self-balancing robots mainly focus on stability using basic sensors and control algorithms, but they differ in efficiency and user interaction. Traditional systems commonly use motor drivers like L298N, which have higher power loss, whereas this project uses the TB6612FNG motor driver for better efficiency and performance. Earlier models often rely on single-sensor configurations, while this system uses dual MPU6050 sensors for improved accuracy in both balancing and gesture detection. In terms of communication, many existing systems use wired or Bluetooth control with limited flexibility, whereas this project employs ESP32 and ESP8266 for faster and more reliable wireless communication. Additionally, most conventional robots lack intuitive control methods, but this system introduces gesture-based

Research Gap

Although significant progress has been made in the development of self-balancing robots, several research gaps still remain in terms of system integration, efficiency, intelligence, and user interaction. Most existing self-balancing robots focus mainly on maintaining stability through PID control and MPU6050-based sensing but lack advanced and intuitive control mechanisms such as gesture-based interaction. While some systems provide wireless operation, they commonly depend on Bluetooth or limited-range communication technologies, reducing flexibility and real-time usability in larger environments.

Another major gap is related to hardware efficiency. Many designs continue to use conventional motor drivers such as L298N, which suffer from high power losses, excessive heat generation, and lower efficiency, ultimately affecting battery life and robot performance. More efficient motor drivers like TB6612FNG are available but remain underutilized in low-cost robotic implementations.

Furthermore, most balancing robots employ a single MPU6050 sensor exclusively for balance estimation and fail to exploit its full capability for additional functions such as gesture recognition and motion-based control. The absence of multi-functional sensor utilization increases system complexity and cost when separate sensors are added for control purposes.

Existing systems also show limitations in adaptive control and real-time responsiveness. Many robots are designed for controlled environments and struggle to maintain stability on uneven surfaces, during sudden disturbances, or under varying load conditions. Additionally, several implementations lack optimized filtering and sensor fusion techniques, leading to noise, drift, and reduced balancing accuracy.

Another research gap lies in system accessibility and educational usability. Advanced self-balancing robots often involve expensive components, complex programming, and sophisticated control algorithms, making them difficult for students and beginners to design, understand, and replicate. Many projects prioritize technical performance over affordability and practical learning value.

Moreover, limited emphasis has been placed on integrating balancing, efficient motor driving, wireless communication, and gesture-based human–robot interaction into a single compact and

METHODOLOGY

3.1. Tools & Technologies Used

- Microcontroller: ESP8266 (NodeMCU) serves as the main processing unit, executing control algorithms, reading sensor data, generating PWM signals for motor control, and enabling wireless communication with efficient power usage.
- IMU Sensor: MPU6050 provides 3-axis accelerometer and 3-axis gyroscope data for tilt angle and motion detection; communicates with ESP8266 via I2C and forms the core of the balancing system.
- Motor Driver: TB6612FNG used to control the speed and direction of DC motors; receives PWM signals from ESP8266 and drives motors accordingly.
- DC Motors and Wheels provide the necessary movement for balancing; respond to control signals to move forward or backward for maintaining stability.
- Power Supply: Rechargeable Battery supplies power to the entire system, including motors, microcontroller, and sensor modules, ensuring portability and continuous operation.
- Voltage Regulation used to provide stable voltage levels required for ESP8266 and other components to ensure reliable performance.
- Control Algorithm: PID Controller used for real-time balancing by calculating error and generating corrective motor actions based on proportional, integral, and derivative components.
- Communication Protocol: I2C used for communication between ESP8266 and MPU6050 sensor for efficient data transfer.
- Software Tools: Arduino IDE used for programming ESP8266, uploading firmware, debugging, and serial monitoring of system performance.
- Programming Language: Embedded C / Arduino used to develop control logic, sensor interfacing, and motor control operations.
- Hardware Components: ESP8266 NodeMCU board, MPU6050 sensor module, motor driver (L298N/L293D), DC motors, wheels, chassis, rechargeable battery, jumper wires, and connectors for complete system integration.

3.2 Architecture

- ESP32 Microcontroller (Robot Unit): Acts as the main control unit of the robot, performing real-time processing of sensor data, executing the PID control algorithm, generating PWM signals for motor control, and coordinating all system operations efficiently.
- ESP32 / ESP8266 Microcontroller (Gesture Unit): Responsible for capturing gesture data from the MPU6050 sensor, processing the motion inputs, and transmitting control commands wirelessly to the robot using Wi-Fi communication.
- IMU Modules (MPU6050 - 2 Units): One sensor is mounted on the robot to measure tilt angle and angular velocity for balancing, while the second sensor is used in the gesture control unit to detect hand movements and convert them into directional commands.
- Motor Driver (TB6612FNG): Interfaces between the ESP32 and DC motors; receives PWM control signals and efficiently regulates motor speed and direction with low power loss and high performance.
- DC Motors and Wheels: Provide mechanical motion of the robot; adjust forward and backward movement dynamically based on control signals to maintain balance and respond to gestures.
- Power System (Rechargeable Battery): Supplies power to all components including microcontrollers, sensors, and motors, ensuring portability and continuous operation of the system.
- Voltage Regulation Module: Maintains a stable voltage level for ESP32, ESP8266, and sensors to prevent fluctuations, ensuring reliable and safe operation.
- Control System (PID Controller): Continuously calculates the error between the desired upright position and the actual tilt angle, generating corrective outputs to stabilize the robot in real time.
- Communication Interface (I2C Protocol): Enables efficient data transfer between ESP microcontrollers and MPU6050 sensors for accurate and fast sensor readings.
- Wireless Communication (Wi-Fi): Facilitates real-time transmission of gesture commands from the controller unit to the robot, enabling responsive and remote operation.
- Chassis Structure: Provides mechanical support and proper alignment of all components, ensuring stability, balanced weight distribution, and smooth movement of the robot.

3.3. Software and Tools:

- **Firmware Development:** Arduino IDE is used for programming the ESP8266 microcontroller in Embedded C/Arduino language, including I2C communication with MPU6050, PWM signal generation for motor control, and implementation of the PID balancing algorithm.
- **Simulation and Testing:** Basic circuit validation and logic testing are performed using tools like Proteus (optional) and serial monitor outputs to verify sensor readings, control signals, and system behavior before full hardware deployment.
- **Version Control:** Git can be used for managing code versions, tracking updates, and maintaining organized firmware development throughout the project lifecycle.
- **Debugging Tools:** Arduino Serial Monitor is used to analyze real-time sensor data, tilt angle calculations, and PID outputs during calibration and testing of the system.
- **Additional Software:** MPU6050 libraries, Wire (I2C) library, and PID control libraries are used for sensor interfacing, data processing, and control implementation, ensuring accurate and efficient system performance.
- **Additional Tools:** Basic tools such as serial plotter (Arduino) are used to visualize sensor data and system response for better understanding and tuning of the control system.

3.4. Hardware:

- **ESP8266 (NodeMCU):** Acts as the central control unit, processing sensor data, executing the PID control algorithm, generating PWM signals, and coordinating all system operations with efficient power management.
- **IMU Sensor (MPU6050):** Provides 3-axis accelerometer and 3-axis gyroscope data to measure tilt angle and angular velocity, which are essential for maintaining balance.
- **Motor Driver (TB6612FNG):** Serves as an interface between the ESP8266 and DC motors, enabling control of motor speed and direction through PWM signals.
- **DC Gear Motors:** Provide sufficient torque and controlled motion required for balancing and quick response to tilt corrections.
- **Wheels:** Attached to the motors to convert rotational motion into linear movement for maintaining stability.
- **Chassis (Frame):** Provides structural support to mount all components securely and ensures proper weight distribution for effective balancing.

- Rechargeable Battery (Li-ion/Li-Po): Supplies power to the entire system, supporting both high-current motor operation and low-power electronics.
- Voltage Regulator (Buck Converter): Converts battery voltage to stable levels (3.3V/5V) required by ESP8266 and other components, ensuring safe operation.
- Power Distribution: Proper wiring and connections are used to distribute power efficiently to motors, microcontroller, and sensor modules.
- Switch Module (Optional): Used to turn the system ON/OFF safely without disconnecting the battery.
- LED Indicators (Optional): Used for basic status indication such as power ON, system activity, or debugging purposes.
- Connecting Wires and Connectors: Used for reliable electrical connections between all components, ensuring proper signal and power flow.
- Mounting Accessories: Includes screws, brackets, and holders to firmly fix components and reduce vibration effects on the sensor.

3.5. Workflow

Start → System powers on and ESP32/ESP8266 initialize all components → MPU6050 sensors (robot unit and gesture unit) begin measuring tilt angle and motion → Gesture MPU6050 detects hand movements → ESP32 (gesture unit) processes gesture data → Gesture commands (forward/backward/left/right) are transmitted wirelessly via Wi-Fi → ESP32 (robot unit) receives gesture commands → MPU6050 on robot measures tilt angle and angular velocity → ESP32 reads sensor data through I2C communication → Sensor data is filtered to obtain accurate tilt angle → Error is calculated between desired upright position and actual tilt → PID controller computes corrective output → ESP32 generates PWM signals for TB6612FNG motor driver → Motor driver controls DC motors based on balancing and gesture inputs → Motors rotate forward or backward to maintain balance and perform movement → Robot attempts to stabilize while responding to gestures → Continuous feedback loop updates sensor data and motor response → If disturbance occurs (push or uneven tilt), system corrects immediately → If gesture input is given, robot changes direction accordingly → If balanced, robot maintains upright position → Power supply ensures continuous operation → System continues balancing and gesture control until power is turned off → Robot stops and system enters idle state

4.IMPLEMENTATION

4.1. Modules Description

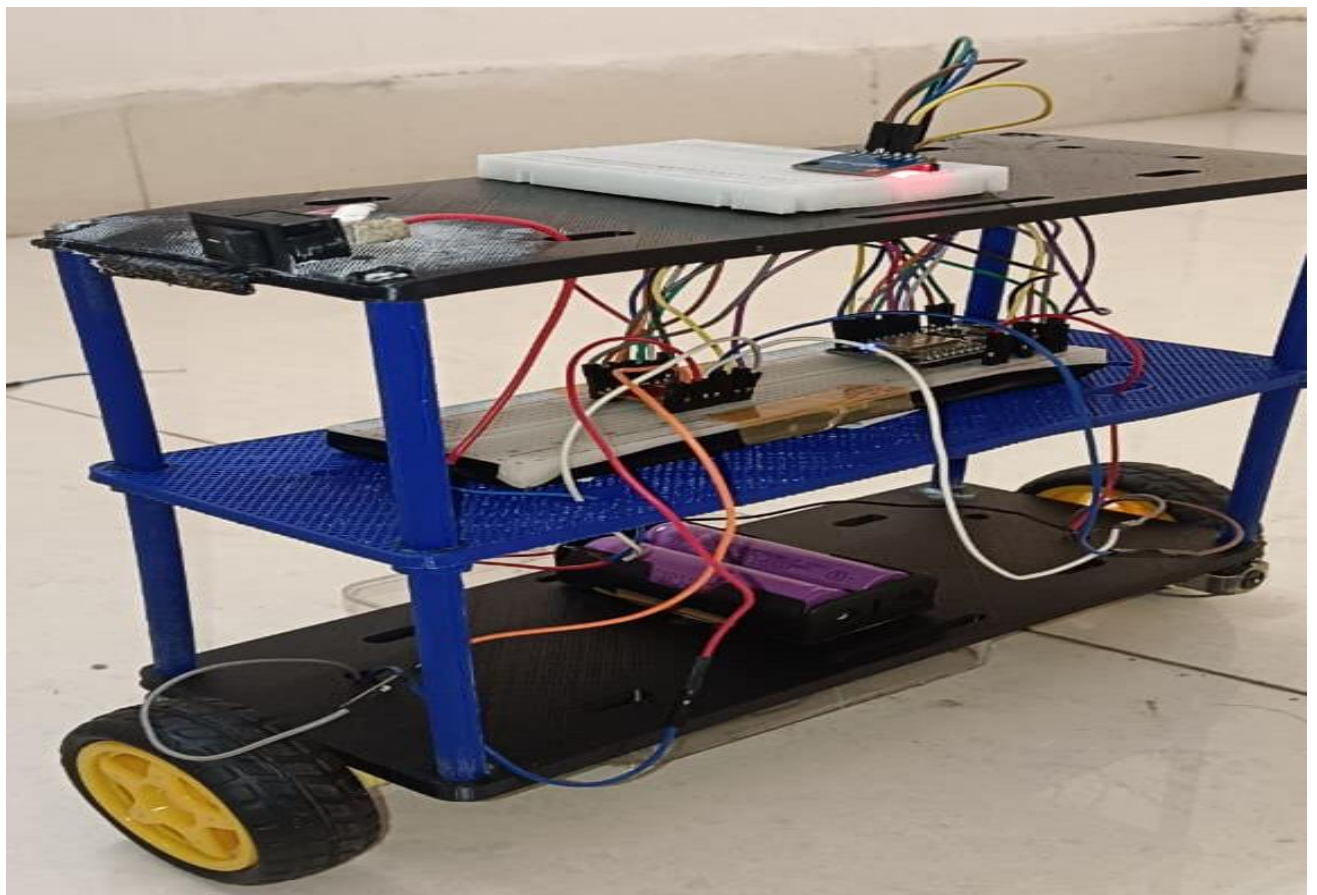
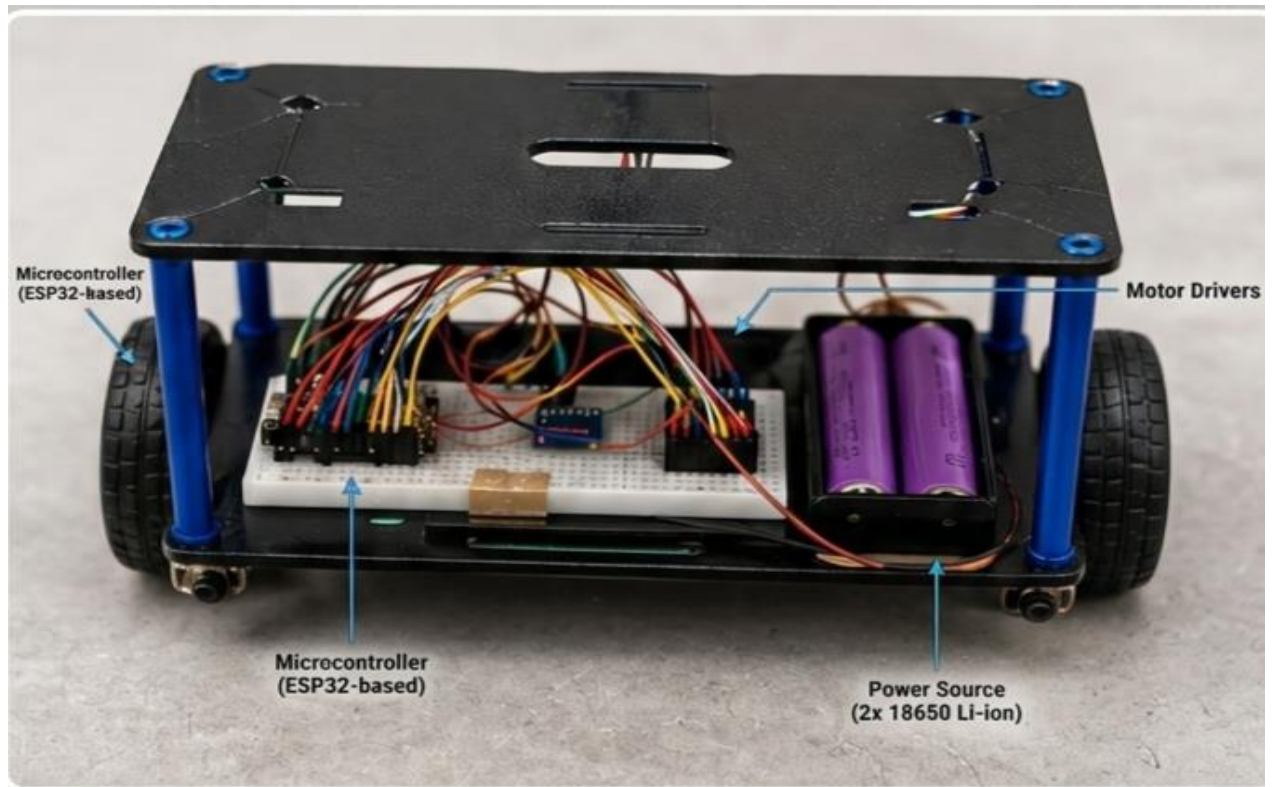
- Control Module: Handles real-time balancing using the ESP8266 microcontroller, processing sensor data, executing the PID control algorithm, and generating PWM signals to maintain stability.
- Sensing Module: Utilizes the MPU6050 IMU to measure tilt angle and angular velocity, providing essential data for determining the robot's orientation.
- Motor Control Module: Uses the motor driver (TB6612FNG) to control the speed and direction of DC motors based on control signals from the ESP8266.
- Actuation Module: Consists of DC motors and wheels that convert electrical signals into mechanical movement to correct tilt and maintain balance.
- Power Management Module: Includes a rechargeable battery and voltage regulator to provide stable and efficient power supply to all system components.
- Feedback Control Module: Implements the PID controller to continuously calculate error and generate corrective actions for smooth and stable balancing.
- Communication Module (Optional): Utilizes the built-in Wi-Fi capability of ESP8266 for future enhancements like remote monitoring and control.
- Structural Module: Consists of the chassis and mounting components that support all hardware and ensure proper alignment and stability of the robot.

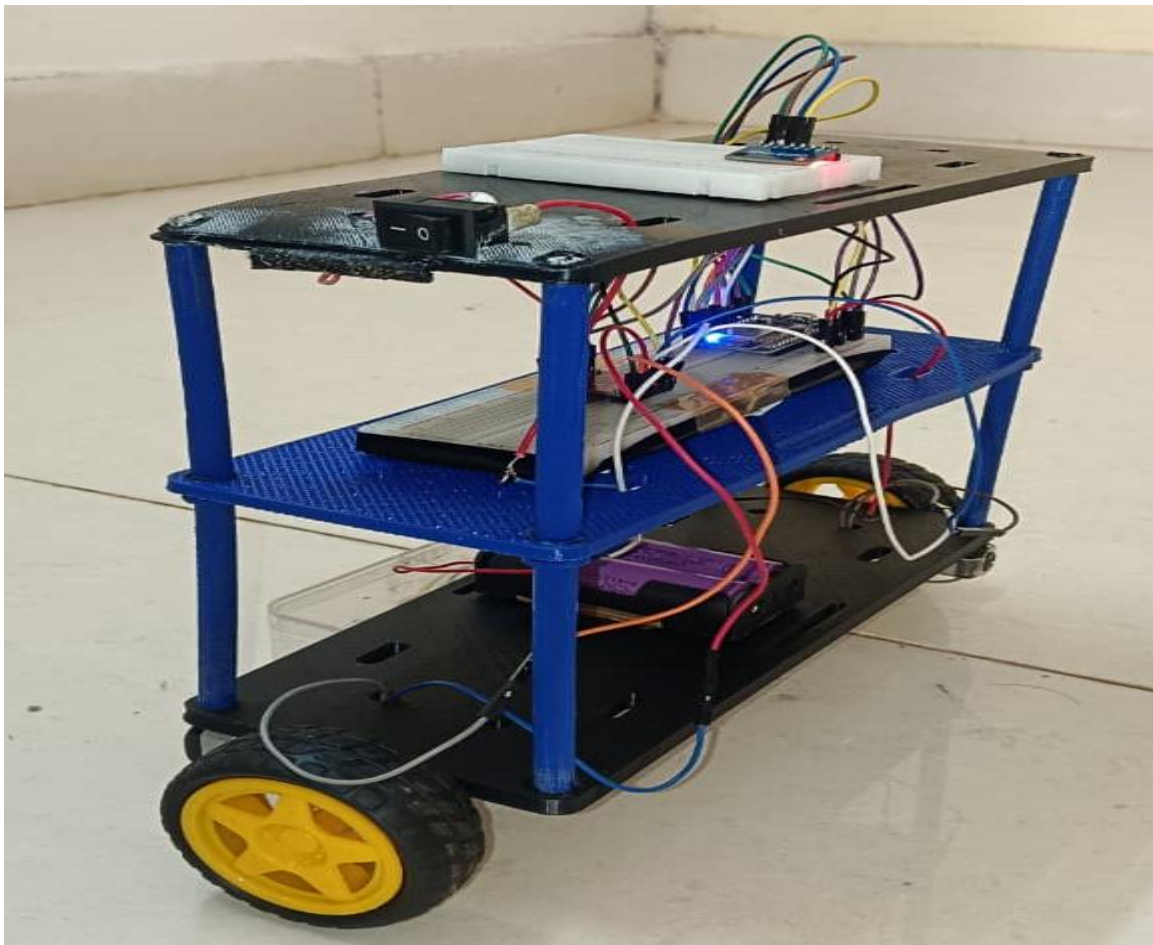
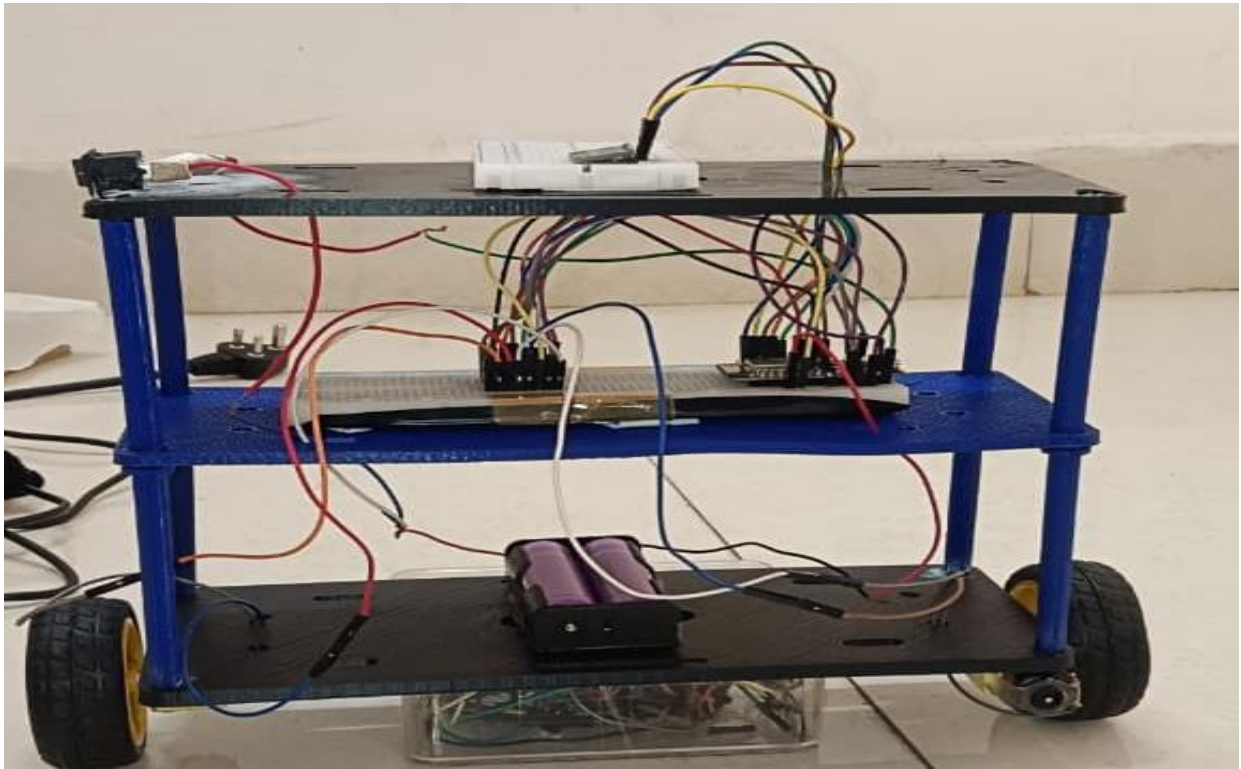
4.2. Algorithms/Processes

- Sensor Data Acquisition: The MPU6050 continuously measures accelerometer and gyroscope data, which is read by the ESP8266 through I2C communication for real-time monitoring of tilt and motion.
- Sensor Data Processing: Raw sensor values are filtered using complementary or basic filtering techniques to obtain accurate tilt angle and reduce noise and drift.
- Error Calculation: The system calculates the difference between the desired position (upright angle) and the actual tilt angle obtained from the sensor.
- PID Control Algorithm: The PID controller processes the error and computes corrective output using proportional, integral, and derivative components to maintain stability.
- PWM Signal Generation: Based on the PID output, ESP8266 generates PWM signals to control motor speed and direction through the motor driver.
- Motor Actuation: The motor driver adjusts the rotation of DC motors, moving the robot forward or backward to correct its tilt.
- Continuous Feedback Loop: The system continuously repeats sensing, processing, and correction in real time to maintain balance.
- Disturbance Handling: If external disturbances such as pushes or uneven surfaces occur, the system detects the deviation and immediately applies corrective action.

5 RESULTS AND DISCUSSION

5.1 Control Panel





6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion:-

The self-balancing robot with gesture control project successfully demonstrates the integration of embedded systems, sensor technology, wireless communication, and control algorithms. The system is capable of maintaining dynamic stability using real-time feedback from the MPU6050 sensor, effectively behaving like an inverted pendulum. The implementation proves that accurate sensing and fast processing are essential for achieving reliable balancing performance. The use of ESP32 and ESP8266 microcontrollers enables efficient data processing and wireless communication between the gesture control unit and the robot. The gesture-based control system enhances user interaction by allowing intuitive movement control without the need for traditional interfaces such as buttons or joysticks. This makes the system more user-friendly and interactive.

The inclusion of the TB6612FNG motor driver improves overall efficiency by providing better motor control with lower power loss compared to conventional drivers. The PID control algorithm plays a crucial role in stabilizing the robot by continuously correcting errors in tilt angle. Proper tuning of the controller ensures smooth and stable operation even under small disturbances.

The project also highlights the importance of system integration, where multiple components such as sensors, microcontrollers, motor drivers, and communication modules work together seamlessly. The robot demonstrates real-time response, stability, and accurate gesture recognition, making it a valuable educational model for understanding robotics and control systems.

In conclusion, this project provides a cost-effective and efficient solution for developing a self-balancing robot with advanced control features. It lays a strong foundation for future improvements such as the addition of artificial intelligence, obstacle detection, and autonomous navigation, expanding its applications in robotics research, automation, and smart mobility systems.

6.2 Future Scope

- Integrate Advanced Control Algorithms: Implement advanced techniques such as Kalman filtering, complementary filters, fuzzy logic, or machine learning-based control to enhance stability, accuracy, and response time beyond traditional PID control.
- Add Obstacle Detection Sensors: Incorporate ultrasonic sensors, IR sensors, or LiDAR modules to enable obstacle detection and avoidance, making the robot safer and more intelligent.
- Enable Autonomous Navigation: Develop path-planning and decision-making algorithms that allow the robot to move independently while maintaining balance in different environments.
- Develop Mobile Application Control: Create a mobile application to control the robot, monitor performance, and tune parameters such as PID values in real-time using Wi-Fi communication.
- Enhance Wireless Communication: Extend the system with IoT capabilities for remote monitoring, control, and cloud-based data analysis using ESP8266/ESP32.
- Improve Sensor Accuracy: Use high-precision IMUs or combine multiple sensors with sensor fusion techniques to reduce noise and improve tilt estimation.
- Enhance Power Management: Implement efficient battery management systems to increase battery life, reduce power consumption, and ensure stable operation.
- Add Data Logging System: Introduce onboard or cloud-based data logging to store sensor readings, system performance, and control responses for analysis and optimization.
- Implement Self-Tuning Mechanism: Develop adaptive algorithms that automatically tune PID parameters based on system performance for improved stability.
- Upgrade Mechanical Design: Improve chassis structure, weight distribution, and motor efficiency to enhance durability, stability, and overall system performance.